

第 2 章 列表和字典

上一章一门课一个变量，三门课还行，三十门课就没法写了。这一章学怎么把一批数据装在一起，做一个能存很多人的通讯录。

2.1 先做一个通讯录

本节任务：把若干人的姓名和电话存起来，能查、能加、能删、能一次列出全部。

```
1 # 一个人用一个字典，多个人装进一个列表
2 contacts = [
3     {"name": "小明", "phone": "13800001111"},
4     {"name": "小红", "phone": "13900002222"},
5 ]
6
7 # 加一个人
8 contacts.append({"name": "小刚", "phone": "13700003333"})
9
10 # 列出全部
11 print(f"共 {len(contacts)} 人")
12 for person in contacts:
13     print(f" {person['name']} {person['phone']}")
14
15 # 按名字查
16 for person in contacts:
17     if person["name"] == "小红":
18         print("查到了: ", person["phone"])
```

运行结果：

```
共 3 人
小明 13800001111
小红 13900002222
小刚 13700003333
查到了:  13900002222
```

这段代码用了两样东西，方括号的**列表**装多个人，花括号的**字典**装一个人的各项信息。它们是 Python 里最常用的两种容器，下面分别讲。

2.2 列表，一串有顺序的数据

本节任务：会建列表、按位置取值、增删改，并知道切片怎么用。

用方括号把若干数据括起来就是列表，里面装什么类型都行，也可以混着装：

```
1 scores = [86, 92, 78]
2 names = ["小明", "小红"]
3 mixed = [1, "两", 3.0, True]
4 empty = []
```

列表是有顺序的，靠**下标**取值，下标**从 0 开始数**。这是新手最容易错的地方，第 ★核心一个元素的下标是 0 不是 1。负数从末尾倒着数，减一就是最后一个：

```
1 scores = [86, 92, 78]
2 print(scores[0]) # 86 第一个
3 print(scores[2]) # 78 第三个
4 print(scores[-1]) # 78 最后一个
5 print(len(scores)) # 3 一共几个
```

下标超出范围会报 `IndexError`，比如上面这个列表访问 `scores[3]` 就会出错，因为最大下标是 2。

增删改都有对应的写法：

```
1 scores = [86, 92, 78]
2 scores[0] = 90 # 改：把第一个换成 90
3 scores.append(65) # 加：在末尾追加
4 scores.insert(1, 100) # 加：插到下标 1 的位置
5 scores.remove(65) # 删：按值删，删掉第一个 65
6 last = scores.pop() # 删：弹出最后一个并返回它
7 print(scores) # [90, 100, 92]
```

取一段连续的元素叫**切片**，写作方括号里两个下标加冒号，规则是**含头不含尾**： ★核心

```
1 nums = [0, 1, 2, 3, 4, 5]
2 print(nums[1:4]) # [1, 2, 3] 从下标 1 到 3，不含 4
3 print(nums[:3]) # [0, 1, 2] 开头省略表示从头开始
4 print(nums[3:]) # [3, 4, 5] 结尾省略表示一直到末尾
5 print(nums[-2:]) # [4, 5] 最后两个
```

含头不含尾看着别扭，但有个好处，`nums[:3]` 和 `nums[3:]` 正好把列表切成不重不漏的两半。

还有一个和列表很像的东西叫**元组**，用圆括号，区别是建好之后不能改。适合存那

些本来就不该变的东西，比如一个坐标：

```
1 point = (3, 4)
2 print(point[0]) # 3
3 # point[0] = 5 # 这一行会报错，元组不能改
```

2.3 字典，按名字取值

本节任务：会用键值对存一个东西的多项信息，并安全地取值。

列表靠位置取值，可要记住第 3 个是电话第 5 个是地址太累了。**字典**换一种方式，给每个值配一个名字，用花括号写，名字和值之间用冒号：

```
1 person = {"name": "小明", "phone": "13800001111", "age": 18}
2 print(person["name"]) # 小明
3 print(person["age"]) # 18
```

冒号左边叫**键**，右边叫**值**。键通常用字符串，而且不能重复。

增删改查：

```
1 person = {"name": "小明"}
2 person["phone"] = "138" # 键不存在就是新增
3 person["name"] = "小明明" # 键已存在就是修改
4 del person["phone"] # 删除
5 print("name" in person) # True 判断键在不在
6 print(len(person)) # 1 有几个键值对
```

取一个不存在的键会报 `KeyError`，这是字典最常见的错误。稳妥的做法是用 `get`，★核心键不存在时返回 `None` 而不是报错，还可以指定一个默认值：

```
1 person = {"name": "小明"}
2 # print(person["phone"]) # KeyError
3 print(person.get("phone")) # None
4 print(person.get("phone", "未填")) # 未填
```

遍历字典有三种方式，看你想要什么：

```
1 person = {"name": "小明", "age": 18}
2 for k in person: # 只要键
3     print(k)
4 for v in person.values(): # 只要值
5     print(v)
```

```
6 for k, v in person.items(): # 键和值一起要
7     print(k, "是", v)
```

最后这种 items 的写法最常用，一次拿到键和值。

2.4 两者怎么配合

本节任务：理解开头通讯录为什么是列表套字典这种结构。

回头看开头那个通讯录。一个人有姓名和电话两项信息，各有各的名字，适合用字典。而人有很多个、有先后顺序、要能追加，适合用列表。所以是**列表里装字典**：

```
1 contacts = [
2     {"name": "小明", "phone": "138"},
3     {"name": "小红", "phone": "139"},
4 ]
5 print(contacts[0]) # {'name': '小明', 'phone': '138'}
6 print(contacts[0]["name"]) # 小明 先按位置取人，再按键取信息
```

这种嵌套非常常见，实际项目里从网上取回的数据大多长这样。取值时从外往里一层层来，contacts 取到列表，加下标取到某个人的字典，再加键取到具体信息。

选哪个容器，问自己一个问题就够了。数据是靠位置区分的还是靠名字区分的。靠位置用列表，靠名字用字典，如表 2.1。

表 2.1: 几种容器怎么选。

容器	写法	什么时候用
列表	方括号	一批同类东西，有顺序，要增删
元组	圆括号	一组不该变的值，比如坐标
字典	花括号加冒号	一个东西的多项信息，按名字取

2.5 一个容易踩的坑

本节任务：知道把列表赋给另一个变量时会发生什么。

看这段代码，它的结果多数人第一次都会猜错：

```
1 a = [1, 2, 3]
2 b = a
3 b.append(4)
4 print(a) # [1, 2, 3, 4] 动的是 b, a 也变了
```

原因是 `b = a` 并没有复制一份数据，只是给同一个列表起了第二个名字。两个名字 ★核心指向同一份数据，从哪个名字改都是改的它。想真正复制一份，用切片或者 `copy`：

```
1 a = [1, 2, 3]
2 b = a[:] # 或者 b = a.copy()
3 b.append(4)
4 print(a) # [1, 2, 3] 这回 a 没变
5 print(b) # [1, 2, 3, 4]
```

数字和字符串没有这个问题，因为它们不能被改。这个坑在后面处理数据时还会遇到，先有个印象。

2.6 小结

这一章学会了成批地存数据。列表用方括号，靠下标取值，下标从 0 开始，切片含头不含尾，能增删改。元组用圆括号，建好不能改。字典用花括号，靠键取值，取不存在的键会报 `KeyError`，用 `get` 更稳妥，遍历常用 `items`。列表套字典是最常见的组合。要注意把列表赋给另一个变量只是起了别名，改一个另一个也会变。下一章用循环和分支把这些数据真正处理起来。

本章知识点

- 列表——方括号括起来的一串数据，有顺序，可以增删改
- 下标——按位置取值，从 0 开始数，负数从末尾倒数
- `IndexError`——下标超出范围，最大下标是长度减一
- 切片——取一段连续元素，含头不含尾
- 元组——圆括号括起来，建好之后不能改
- 字典——花括号加冒号，用键取值，键不能重复
- `KeyError`——取了不存在的键，用 `get` 可以避免
- `get`——取值时键不存在返回 `None` 或指定的默认值，比方括号稳妥
- `items`——同时遍历键和值，字典最常用的遍历方式
- 列表套字典——一批东西每个都有多项信息时的常用结构
- 别名——把列表赋给另一个变量只是起第二个名字，改一个另一个也变